AQI Predictor: End-to-End Serverless Forecasting for Karachi

Maha Asim

June 7, 2026

1 Introduction

The AQI Predictor is a serverless system that forecasts the Air Quality Index for the next three days at hourly resolution for Karachi, Pakistan. Data is fetched from the Open-Meteo API, features are stored in MongoDB Atlas, and predictions are served through a Flask API and Streamlit dashboard deployed to the cloud. The live dashboard is available at https://aqi-predictor-7xvbhqndwxemfcrmqcqz44.streamlit.app.

The forecasting approach is iterative. A model trained to predict AQI one hour ahead is called 72 times in sequence, with weather features updated from the Open-Meteo 3-day forecast and lag features recomputed from the growing prediction history at each step. This produces genuinely varying predictions across all three days.

2 System Architecture

The pipeline has four stages. The data ingestion stage uses `backfill.py` to seed 90 days of historical data via the Open-Meteo Archive API, and `feature_pipeline.py` runs hourly via GitHub Actions to append the latest record, while `training_pipeline.py` runs daily. The feature store is a MongoDB Atlas collection called `aqi_features` which stores one document per hour containing raw pollutants, weather, and all engineered features. The model registry is also on MongoDB Atlas using GridFS to store serialised model artifacts alongside metrics, feature schema, training timestamp, and an `is_best` flag that the backend uses to load the correct model at startup. The serving layer consists of a Flask REST API and a Streamlit dashboard that shows current AQI, the 3-day forecast chart, pollutant breakdown, SHAP feature importance, and 7-day historical trend.

3 Data and Exploratory Data Analysis

The dataset contains 2,467 hourly records from 22 February to 5 June 2026. AQI ranges from 32 to 161 with a mean of 80.2 and standard deviation of 19.6. Around 82% of hours are classified as Moderate.

EDA found that lag and rolling features are the strongest predictors. `aqi_lag_1h` has Pearson $r = 0.97$ with the target, meaning recent AQI history is by far the most informative signal. Among pollutants, $PM_{2.5}$ ($r = 0.65$) and PM_{10} ($r = 0.51$) are most correlated

with AQI. A residual analysis by hour found systematic under-prediction of 2.97 AQI points at 17:00 and over-prediction of 4.23 points at 21:00, indicating an unexploited evening rush pattern. PM_{10}, CO, $PM_{2.5}$, and NO_2 are all strongly right-skewed, justifying log transformation.

A data quality issue was also found during EDA: 75% of weather values in MongoDB were zero. The backfill had used the Open-Meteo forecast endpoint for historical dates, which returns null for past hours, and `value or 0` in Python silently converted those nulls to zeros. The fix was to switch to the Open-Meteo Archive endpoint and replace the null handling with an explicit check. A re-backfill overwrote all affected records.

4 Feature Engineering

36 features were built in total. The raw pollutants $PM_{2.5}$, PM_{10}, NO_2, O_3, CO, and SO_2 were included directly. Log versions of the skewed ones were added since their distributions were heavily right-skewed. Weather features temperature, humidity, pressure, and wind speed were included after fixing the zero-value issue. Two wind interaction features were created by dividing $PM_{2.5}$ and PM_{10} by wind speed, since higher wind disperses pollution and EDA confirmed these had useful correlation with AQI.

For AQI history, lag values were created at 1, 2, 3, 6, 12, 24, and 48 hours back, along with rolling averages at 3, 6, 12, and 24 hour windows. A 3-hour and 6-hour momentum feature was also added to capture whether AQI is rising or falling. A rolling standard deviation over 6 hours was included to tell the model whether AQI is currently stable or volatile.

For time, hour and month were encoded as sin and cos values rather than raw numbers, since hour 23 and hour 0 are adjacent in reality but would appear far apart as integers. A binary flag called `peak_traffic_hour` was set to 1 for hours 15 to 20 after residual analysis showed the model was consistently under-predicting during evening rush hour.

Three features were removed after EDA showed they added no value. The humidity interaction terms had near-zero correlation because humidity was mostly zeros during the initial backfill. Day of week had $r = 0.03$ and AQI change rate had $r = 0.12$, both too weak to be useful.

5 Model Training and Evaluation

Five models were trained: Ridge, Random Forest, Extra Trees, XGBoost, and Gradient Boosting, plus a stacking ensemble. Hyperparameter search used TimeSeriesSplit with 5 folds to prevent future data from leaking into validation. The dataset was split 80/20 chronologically for the final holdout.

Table 1: Model Results

Model	Holdout			TimeSeriesCV		
	RMSE	MAE	R^2	RMSE	MAE	R^2
Ridge	12.99	2.87	0.404	—	—	—
Random Forest	8.30	2.03	0.757	7.28	3.45	0.832
Extra Trees	8.09	1.99	0.769	7.19	3.46	0.838
XGBoost (winner)	8.64	2.21	0.737	6.06	2.84	0.876
Gradient Boosting	8.44	2.08	0.749	6.35	3.03	0.868
Stacking	8.70	2.45	0.733	6.87	3.92	0.849

XGBoost was selected with a TimeSeriesCV R^2 of 0.876 and RMSE of 6.06. The holdout R^2 of 0.737 is lower because the test window covers Karachi's pre-monsoon transition where AQI dropped to a mean of 59.6 one week then spiked to 87.9 the next, a pattern the model had not seen during training. The TimeSeriesCV score averaged across five windows is the more reliable estimate.

6 SHAP Interpretability Analysis

SHAP was applied using TreeExplainer on 500 recent observations. The top features by mean absolute SHAP value were `aqi_lag_1h` (9.51), `aqi_rolling_12h` (1.17), `aqi_lag_6h` (1.02), `aqi_momentum_3h` (0.99), `aqi_lag_3h` (0.89), `pm2_5` (0.73), `aqi_lag_2h` (0.65), `aqi_std_6h` (0.55), `aqi_rolling_6h` (0.44), and `o3` (0.38).

Lag and rolling features account for 73.1% of total predictive power, time and other features 12.0%, pollutants 9.4%, and weather 5.5%. The dominance of `aqi_lag_1h` at more than eight times the next feature is consistent with the autocorrelation of 0.97 found in EDA. $PM_{2.5}$ ranking sixth confirms it adds real signal beyond temporal history alone. `aqi_std_6h` ranking eighth validates the volatility feature.

7 Challenges and How They Were Overcome

The first challenge was that all three days returned the same AQI. The forecast loop was updating only time features at each step while weather and pollutants were frozen. The fix was to fetch the 3-day Open-Meteo forecast at prediction time and inject the correct values at each of the 72 steps, with lag features recomputed from growing prediction history.

The second challenge was weather data being silently zero for 75% of records, diagnosed through EDA. The cause was use of the forecast endpoint for historical dates combined with Python's `or 0` idiom converting nulls to zeros. Switching to the archive endpoint and using explicit null checks resolved it.

The third challenge was the model performing worse than a naive predictor. The naive strategy of returning `aqi_lag_1h` directly achieves RMSE 4.35 while the model achieved 8.31. A residual analysis by hour identified systematic errors during evening hours the model was missing. The response was to switch to TimeSeriesCV, add the `peak_traffic_hour` flag, `aqi_std_6h`, and `aqi_lag_48h`.

The fourth challenge was XGBoost failing inside StackingRegressor due to a version incompatibility with scikit-learn. XGBoost was replaced in the stack with Extra Trees and kept as a standalone model, where it won selection.

8 Conclusion

The project delivers a working serverless AQI forecasting system with 72 genuinely varying hourly predictions, a TimeSeriesCV R^2 of 0.876, EDA-driven feature engineering, full SHAP explainability, and cloud infrastructure across MongoDB Atlas, GitHub Actions, Render, and Streamlit Community Cloud. The holdout R^2 of 0.737 reflects an anomalous test window rather than a fundamental model limitation and will improve as the system accumulates data across Karachi's full annual weather cycle.